Sketchpad: Architecture Rethink

Discussion, 1 November 2026. Nothing here is decided or built. Item numbers such as 1.3 or 4.2 refer to the list in `docs/NOT_DONE.md` on branch `claude/batch-1`.

What Sketchpad would look like if rebuilt from scratch, with the items in `docs/NOT_DONE.md` folded in. Nothing here is decided or built. Existing scripts need not stay compatible, as long as each can be rewritten.

The core change: make the figure the center of the app, and make an object a plain description that produces geometry as data. Today an object is a stateful thing that draws itself. Most of the awkward code exists because of that, and so do most of the not-done items.

6.1 An object becomes a recipe

Today every registry object has `params`, `init`, `update`, `draw` and an `elements` cache. A Line keeps its end points twice: as `pt1`, `pt2` and `midpoint`, and again in `points[]` for the handles. `update()` then works out which one was dragged and copies them back and forth.

Proposed: an object is

- **params**: plain values, including named points such as `pt1` and `pt2`.
- **handles**: a list naming which params can be dragged.
- **build(params)**: returns geometry, with no side effects.

Dragging a handle writes the named param directly. There is no `points[]` mirror and no syncing in `update()`.

What this removes:

- **The shared-settings bug (Not Done 4.1).** With no singletons holding state, every use is its own copy.
- **Guesswork in the weld code.** `offsetParams()` guesses where an object keeps its positions, and `findHandleIndex()` matches handles by comparing coordinates. Both exist only because handles are not named.
- **`decompose()` and `center()` (2.4, 2.6).** If the geometry is already a list of pieces, break apart just takes the pieces. The center is computed from the geometry or declared by the object.

6.2 Geometry is data, and drawing is one function

`build()` returns something like:

```
{ segments: [ {a, b, role: "stitch"}, ... ],
  arms:     [ {id, a, b, steps} ],          // what infrastructure() returns today
  points:   { corner1: {x,y}, apex: {x,y} } // named points; magic points live here
}
```

One renderer draws any geometry. The current special cases become options:

- **Final mode, points mode, dimming, layer transforms.** `drawState.final` and `drawState.transform` go away.
- **Bounds.** Computed from the geometry, not noted as a side effect while drawing.
- **Printing at size (Phase 9).** The same geometry goes to SVG or PDF at real scale.
- **Nail spacing.** Measuring distances over a list of points.
- **Thumbnails.** The same renderer drawing smaller.

Tradeoff for Barry to rule on. Making every object produce geometry is "forcing every object through one drawing path", which CLAUDE.md says to raise. `drawSpirals.js` (24 direct canvas calls) and the circle and text helpers would have to produce segments or curves. Nothing is dropped: Circular Parabola (Multi) and Linked Circles stay as playful. But that code is rewritten.

6.3 Points, discs, welds and magic points become one idea

Today: three kinds of disc (anchored, free, handle), weld groups keyed by disc numbers, `cornerPartners()` to tie a Parabola's two corner handles, a propagation walk with four special cases, and 8b's magic points still to come.

Proposed, two concepts:

- **A point reference**: a named point of an object, a spot along an arm (arm plus fraction t), or a free point owned by the figure. A magic point is just another named point.
- **A weld**: a constraint saying "these points coincide". When you drag, a small solver moves what is free to move. Rules decide which side moves (the dragged side drives, as now).

A weld group is then one nail, which is what Phase 9 needs. The red mark for a weld that cannot be satisfied becomes "the solver could not satisfy it". Corner tying disappears: a Parabola corner is one named point shared by both arms.

This changes behaviour, not just code. The 6b/6c rules (discs do not drag, only handles do) were set on purpose. A solver can keep them, but it is a choice to make again, not something carried over automatically.

6.4 One kind of document, with the tabs as views of it

Draw is a figure with one object. A saved Draw setting (today's "secondary object") is a saved figure. Patterns are scripts. The six tabs can stay, but these shared parts get built once instead of in parallel: the Draw and Figures save paths, `drawMenuCmds.js` and `figuresCommands.js`, and the separate restore code in each tab.

6.5 State has one owner, and changes go through named actions

Today `uiState`, `drawState` and `ctx` are bare globals. Each tab has its own `save()` and `restore()`, and none of the `save()` functions is ever called. There are 46 `.style.display` lines and 46 `!important` rules.

Proposed: one state object (still declared, still written through immediately) and every change through a named action: `insertLayer`, `moveHandle`, `weld`, `deleteLayer`. That gives:

- **Undo**: keep the list of actions or snapshots. Phase 10's title promised undo (1.3).
- **Unsaved-change tracking**: today worked out by comparing snapshots.
- **Visibility read from state**: each region redraws itself from state, replacing the show/hide lines (4.5 to 4.7).
- **Simpler tab activation**: one way to restore a tab (3.4, 3.6).

Plain hand-written code, not React or another framework. Each region gets a `render(state)` function that reads top to bottom.

6.6 One table of commands

Each command is `{id, label, enabledWhen(state), run()}`, used by the caption menu, the Commands panel and the keyboard. This is the figuresCommands pattern (Phase 15, 1.8) applied to every tab.

6.7 Scripts get an API instead of globals

Each of the 195 scripts exports a title, its controls, and a function that receives a drawing API (lines, curves, points, text). No `window.armInteractor`, no shared `drawState`, and each run starts from scratch, which fixes the state leak (4.2). Because a script produces the same geometry as a registry object, promoting one becomes moving a file and tidying it.

This is the biggest job in the list. Many scripts follow the same few patterns, so much of it is mechanical. Each is checked against a picture of its current output.

6.8 Files: one storage module and simpler indexes

Phase 14 (1.7), done from the start. The app calls named operations (`loadIndex`, `saveFigure`, `writeThumbnail`); only the storage module builds paths or Node requests.

Indexes: today a manifest per category, plus `home/manifest.json`, plus directory registries, plus a start-up rebuild and a missing-file check. Proposed: one small metadata file per item (title, status, tags) and a single index built from the folders. Status lives with the item, so an index cannot drift from the folders.

6.9 Real units from the start

The document stores millimetres or inches, with the canvas as a zoomed view. The end goal is a template printed at size; today coordinates are canvas pixels, so Phase 9 would have to add a scale on top.

6.10 Templates, scaffold and background images are layer roles

A layer's role is stitch, guide, scaffold or image. Guides and scaffold never print. Scaffold is one-way: things attach to it, not the reverse. A background image is another role. That settles the template X (2.1, 2.2), "release from scaffolding", and background image import with one field.

6.11 Help in Markdown, edited in VS Code

Dropping TinyMCE removes about 148,000 lines of someone else's code and the stale-skin problem (T17). The Notes popup keeps reading the header comments.

6.12 Tests split in two

`build()` takes params and returns geometry with no side effects, so geometry is tested in Node in milliseconds, with no browser. Each object gets a fixed picture to compare against. The browser suite shrinks to UI wiring.

6.13 What stays

- The two purposes, and the right to keep odd objects around for play.
- Stable layer numbers, with stacking order kept separate.
- The layer transform model.
- State written through immediately, with globals declared.
- Header comments that say what a file does.
- The local Node service for writes, and the no-ad-hoc-divs rule.
- One choke point for drawing. It moves from `drawLine` to the geometry, but the idea is the same.

6.14 Where the items above land

Item	In the rethink
1.1 8b magic points	Named points from `build()` (6.1, 6.3)
1.2 Phase 9 print aids, nail spacing	SVG/PDF from geometry in real units; weld group = one nail (6.2, 6.3, 6.9)
1.3 Phase 10 undo; 2.10 saving welds	Action log; welds are constraints in the document (6.3, 6.5)
1.4 Phase 11 figure follow-ups	Mostly fall out of the action layer (6.5)
1.5 Phase 12 Insert as icons	The shared renderer draws them (6.2)
1.6 Phase 13 sweep tool	Still needs a definition; an action acting on geometry
1.7 Phase 14 filesystem boundary	The storage module (6.8)
1.8 Phase 15 commands pattern	The commands table (6.6)

Item	In the rethink
2.1, 2.2, 2.3 templates, X; scaffold; background image	Layer roles (6.10)
2.4 composite break apart; 2.6 `center()`	Not needed; geometry is already pieces (6.1)
2.5 curved-arc primitive	A curve kind in the geometry (6.2)
2.7 parametric draggable points	Named points work the same for parametric objects (6.1)
2.8, 2.9, 2.11, 2.12 weld and disc questions	Decided once, in the solver rules (6.3)
2.13 weld group marker	One nail = one mark (6.3)
2.14 `printCircNum`, `drawState.ctr`	Gone with the script API (6.7)
3.2 `validateDrawRegistry`	A schema check on the object format (6.1)
3.4, 3.6 tab activation, restore shape	One restore path (6.5)
4.1 shared object settings	Gone; no singletons (6.1)
4.2 script state leak	Gone; fresh run per script (6.7)
4.5, 4.6, 4.7 `!important`, show/hide, fixed divs	Regions render from state (6.5)
4.8 script problems	Found and fixed while porting (6.7)
4.12, 4.13 help pages, PDFs	Markdown help (6.11)

6.15 Recommendation: rebuild in stages, not all at once

A full rewrite of about 27,000 lines plus 195 scripts would take months, and nothing would work in the meantime. Instead, stages inside the running app, each its own phase with a hold between them:

1. The new object format, with an adapter so old registry objects still run. Port Line and Parabola first.
2. Named points and welds as constraints, replacing discs, `cornerPartners()` and the propagation walk. Covers 8b.
3. One state owner and named actions, with undo.
4. The commands table, tab by tab (Phase 15).
5. The storage module and per-item metadata (Phase 14).
6. The script API, then porting scripts in batches.
7. Real units and print output (Phase 9).

Stages 1 to 3 are the ones that pay off; the rest is tidying that could stop short. The cost is the same as a rewrite, but the app keeps working throughout, and work can stop after any stage.